



Deep Learning

Tutorial 12: Basic Auto Encoders

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

Improve the Model:

Improve the autoencoder model by experimenting with different architectures (e.g., adding more layers, using convolutional autoencoders) to improve the reconstruction quality.

Implementation of these tasks are in PyTorch:

Convolutional Autoencoder for MNIST Image Reconstruction

1. Introduction

This Tutorial task focuses on designing and evaluating a convolutional autoencoder for reconstructing handwritten digit images from the MNIST dataset. Autoencoders are unsupervised learning models that compress input data into a lower-dimensional representation and then reconstruct it back into its original form. The objective of this work was to create a model capable of generating sharper and more accurate reconstructions than a basic dense autoencoder. To achieve this, a deeper U-Net style convolutional architecture was implemented, allowing the model to retain fine image details during the reconstruction process.

2. Dataset Description

The MNIST dataset contains sixty thousand training images and ten thousand test images, all of which are grayscale images with a resolution of twenty-eight-by-twenty-eight pixels and a single channel. Each image contains one handwritten digit from zero to nine. Before training, the pixel values were normalized to a range between zero and one to stabilize the training process and improve the learning efficiency. The images were reshaped into a four-dimensional format of (number of samples, 28, 28, 1), making them compatible with convolutional neural network layers.

3. Model Architecture

A U-Net convolutional autoencoder architecture was developed to improve reconstruction clarity. The encoder portion of the model consists of multiple convolution layers followed by batch

normalization and ReLU activation, which help extract progressively deeper image features. Max pooling layers reduce spatial dimensions, allowing the model to learn compact feature representations. At the bottleneck stage, the model uses one hundred twenty-eight filters to encode rich latent information. The decoder then upsamples the compressed representation using transposed convolution layers and gradually reconstructs the original image size. Skip connections are used throughout the decoder, allowing the model to directly reuse feature maps from the encoder. This greatly improves the sharpness and structural accuracy of the reconstructed images. The final layer uses a single-filter convolution with a sigmoid activation to produce the output image. The complete model contains approximately four hundred sixty-eight thousand parameters.

4. Training Procedure

The model was trained in Google Colab using TensorFlow and Keras. The Adam optimizer was selected because of its strong performance in deep learning applications, while binary cross-entropy was used as the loss function due to the normalized pixel range. The training was performed for twenty epochs with a batch size of one hundred twenty-eight, and both training and validation losses were monitored throughout the process. The model demonstrated stable convergence and did not show any symptoms of overfitting or instability. The final recorded training loss was approximately 0.0584, while the validation loss was approximately 0.0581. These values indicate that the autoencoder learned to reconstruct the images effectively and consistently.

5. Results

The reconstructed images produced by the model were noticeably sharper and more detailed than those from a simple dense autoencoder. The presence of skip connections allowed the model to preserve structural features of digits such as curvature and thickness. The visual results show that the reconstructed outputs closely resemble the original inputs, with clear digit outlines and minimal distortion. The training and validation loss curves also display smooth downward trends, confirming that the model was learning effectively throughout the process. Overall, the results demonstrate that the U-Net autoencoder design successfully enhances reconstruction quality.

6. Discussion

The experiment demonstrates the advantages of using convolutional architectures with skip connections for image reconstruction tasks. The deeper encoder-decoder structure enables the model to capture both global and local features, leading to improved sharpness in the output images. The steady convergence of loss values suggests that the hyperparameters, architecture, and optimization method were appropriately selected. Although the MNIST dataset is relatively simple, the model performed consistently well and produced high-quality reconstructions. However, because MNIST does not contain complex textures or high-resolution features, future experiments on more challenging datasets would provide a stronger assessment of the model's generalization capabilities. Additional enhancements such as perceptual loss or denoising objectives may further improve reconstruction sharpness.

7. Conclusion

In conclusion, a U-Net based convolutional autoencoder was successfully implemented and trained on the MNIST dataset. The model achieved strong reconstruction accuracy and produced visually sharp outputs while maintaining low training and validation losses. The architecture effectively captured relevant image features and demonstrated the benefits of skip connections for preserving detail during reconstruction. This experiment confirms that U-Net style autoencoders are well-suited for tasks involving image recovery, compression, and enhancement. Further improvements may be explored through the use of alternative loss functions, deeper latent spaces, or advanced generative models such as variational autoencoders or GAN-based autoencoders.

8. Full code

```
import numpy as np

import matplotlib.pyplot as plt

from keras.layers import Input, Conv2D, MaxPooling2D, Conv2DTranspose, concatenate,
BatchNormalization, ReLU

from keras.models import Model
```

```

from keras.datasets import mnist

%matplotlib inline

# -----

# 1. Load & prepare MNIST

# -----

(XTrain, _), (XTest, _) = mnist.load_data()

XTrain = XTrain.astype("float32") / 255.
XTest = XTest.astype("float32") / 255.

XTrain = np.expand_dims(XTrain, -1)
XTest = np.expand_dims(XTest, -1)

print("Train:", XTrain.shape, "Test:", XTest.shape)

# -----

# 2. U-Net blocks

# -----

def conv_block(x, filters):

    x = Conv2D(filters, 3, padding='same')(x)

```

```
x = BatchNormalization()(x)
```

```
x = ReLU()(x)
```

```
x = Conv2D(filters, 3, padding='same')(x)
```

```
x = BatchNormalization()(x)
```

```
x = ReLU()(x)
```

```
return x
```

```
def enc_block(x, filters):
```

```
    c = conv_block(x, filters)
```

```
    p = MaxPooling2D((2,2))(c)
```

```
    return c, p
```

```
def dec_block(x, skip, filters):
```

```
    x = Conv2DTranspose(filters, 2, strides=2, padding='same')(x)
```

```
    x = concatenate([x, skip])
```

```
    x = conv_block(x, filters)
```

```
    return x
```

```
# -----
```

```
# 3. Build U-Net Autoencoder
```

```
# -----
```

```

inp = Input(shape=(28, 28, 1))

# Encoder

c1, p1 = enc_block(inp, 32) # 28→14
c2, p2 = enc_block(p1, 64) # 14→7

# Bottleneck

bn = conv_block(p2, 128)

# Decoder

d1 = dec_block(bn, c2, 64) # 7→14
d2 = dec_block(d1, c1, 32) # 14→28

out = Conv2D(1, 1, activation="sigmoid")(d2)

autoencoder = Model(inp, out)

autoencoder.summary()

autoencoder.compile(optimizer="adam", loss="binary_crossentropy")

# -----

# 4. Train

```

```

# -----

history = autoencoder.fit(

    XTrain, XTrain,

    epochs=20,

    batch_size=128,

    validation_data=(XTest, XTest)

)

# -----

# 5. PLOT TRAINING LOSS

# -----

plt.figure(figsize=(7,4))

plt.plot(history.history['loss'], label='Train Loss', linewidth=2)

plt.plot(history.history['val_loss'], label='Val Loss', linewidth=2)

plt.title("Training & Validation Loss", fontsize=14)

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid(True)

plt.show()

# -----

```



```
# 6. PREDICT RECONSTRUCTIONS
```

```
# -----
```

```
decoded = autoencoder.predict(XTest)
```

```
# -----
```

```
# 7. SHOW ORIGINAL vs RECON
```

```
# -----
```

```
n = 10
```

```
plt.figure(figsize=(20,4))
```

```
for i in range(n):
```

```
    # original
```

```
    ax = plt.subplot(2, n, i+1)
```

```
    plt.imshow(XTest[i].reshape(28,28), cmap="gray")
```

```
    ax.axis("off")
```

```
    # reconstructed
```

```
    ax = plt.subplot(2, n, i+1+n)
```

```
    plt.imshow(decoded[i].reshape(28,28), cmap="gray")
```

```
    ax.axis("off")
```

```
plt.show()
```

```
# -----  
  
# 8. OPTIONAL: Histogram of pixel intensities  
  
# -----  
  
plt.figure(figsize=(7,4))  
  
plt.hist(decoded.reshape(-1), bins=50, color='purple')  
  
plt.title("Distribution of Reconstructed Pixel Values")  
  
plt.xlabel("Pixel Intensity")  
  
plt.ylabel("Frequency")  
  
plt.grid(True)  
  
plt.show()
```